

Industrialiser et gouverner l'IA générative

Du protocole MCP à l'usine logicielle IA

Monter une platform team IA, ce n'est pas écrire un agent de plus : c'est construire la « route pavée » que les équipes produit du groupe emprunteront — un socle d'intégration fiable (MCP), un contexte LLM partagé et gouverné, et une discipline d'évaluation qui tient en production.

Cette formation parcourt cette chaîne de bout en bout, avec un fil rouge concret et « méta » : les participants outillent la fabrication même des serveurs MCP. Les travaux pratiques sont menés en Java / Quarkus, pilotés avec l'agent OpenCode, versionnés et gouvernés sur GitLab, tout en restant agnostiques sur le fournisseur de LLM. Deux jours suffisent à **poser les fondations** ; ils ne livrent pas une plateforme prête pour la production.

Informations clés

Durée	2 jours (14 heures) : le parcours essentiel, avec un environnement préparé avant la séance. Les approfondissements des guides sont complémentaires, hors de ces 14 heures.
Référence	IA-PLT-USINE — sur-mesure / intra
Public	Platform engineers, tech leads, architectes, profils DevEx/DevOps, référents IA
Prérequis	Savoir lire et compléter un test Java simple (JUnit) · bases de Git et Maven · expérience d'intégration d'une API LLM · notions d'IA générative · poste préparé avant la séance selon la checklist fournie (JDK 21+, Maven, OpenCode connecté)
Conditions techniques	Fournisseur de LLM autorisé par l'organisation (génération et juge du TP3) · accès réseau vers ce fournisseur et les dépôts Maven · projet GitLab d'atelier avec un runner adapté. Le catalogue métier est autonome ; la formation n'est pas hors ligne.
Modalités	Présentiel · alternance d'apports et de travaux pratiques · guides pas à pas en ligne pour chaque TP
Évaluation	Travaux pratiques continus et grille d'auto-évaluation avant/après sur quatre compétences (ci-dessous) · attestation remise

Objectifs pédagogiques

- Situer l'écosystème agentique et le rôle de MCP comme socle d'intégration au système d'information
- Piloter un agent (OpenCode) pour produire un serveur MCP en Quarkus, et savoir en relire et valider la sortie
- Distiller un savoir-faire dans un Skill et partager le contexte LLM entre équipes de façon gouvernée
- Définir une stratégie d'évaluation des produits IA et l'outiller en Java
- Poser les fondations d'une gouvernance IA et d'un modèle opératoire d'usine logicielle IA à l'échelle du groupe

Fil rouge

Un même produit traverse les deux jours : le Skill `create-quarkus-mcp-server`, qui génère des serveurs MCP conformes. On fait d'abord produire un serveur « vanilla » par OpenCode, on distille ce savoir-faire dans un Skill, on l'évalue avec un harnais Java, puis on le gouverne sur GitLab. Le domaine support — un catalogue de services interne, auto-contenu — assure une exécution fiable en salle.

Programme

Jour 1 — Construire et partager

- M0 · La ligne de flottaison (keynote)** — le modèle mental qui structure les deux jours : ce que la plateforme construit, ce que le socle finit par absorber.
- M1 · Fondations agentiques & protocoles** — rappel du paysage LLM, anatomie d'un agent, tools / skills / mémoire, panorama MCP / A2A / A2UI / AP2.
- M2 · Architecture MCP** — hôtes, clients, serveurs ; primitives tools / ressources / prompts ; transports ; implémentation en Quarkus.
- TP1 · Serveur MCP « vanilla » avec OpenCode** — piloter l'agent, relire la sortie ; comparaison des générations et mesure des écarts éventuels — une baseline déjà conforme est un résultat valable.
- M3 · Le contexte LLM comme actif partagé** — context engineering ; MCP (outils) vs Skills (savoir-faire) ; OpenCode comme consommateur agnostique.
- TP2 · Écrire le Skill create-quarkus-mcp-server** — distiller la conformité, puis « dogfood » : régénérer un serveur et le comparer à la baseline.

Jour 2 — Évaluer et gouverner

- M4 · Évaluer les produits IA** — spécificités de l'évaluation LLM, stratégie, datasets (happy / realistic / adverse), scorers ; lecture des résultats.
- TP3 · Évaluer le Skill** — harnais d'évaluation 100 % Java (JUnit + LLM-as-a-Judge) sur le catalogue de services ; preuve qu'un test sait échouer.
- M5 · Exploiter & sécuriser en production** — observabilité, guardrails, sécurité des accès MCP (pattern gateway) ; mini-exercice de lecture d'une trace.
- M6 · Gouverner le contexte à l'échelle** — GitLab : versioning, permissions, données sensibles, drift.
- TP4 · Gouverner le Skill sur GitLab** — merge requests, branch protection, évaluation en intégration continue, permissions par agent.
- M7 · Atelier — le modèle opératoire de l'usine logicielle IA** — catalogue de capacités, cycle de vie, fiche de sortie pour une première capacité du groupe.

Évaluation des acquis

Elle porte sur les **compétences des participants**, distinctes de la note obtenue par le Skill dans le harnais : la réussite du modèle ne vaut pas compétence acquise. Chacun se positionne en début puis en fin de formation sur quatre capacités observables :

Compétence	Ce qui est observé
Lire un contrat MCP	Identifier tools, ressources et prompts exposés par un serveur et vérifier leur contrat avec un inspecteur
Comparer une génération	Mesurer les écarts d'une sortie d'agent à une convention, avec preuves, sans présumer un échec
Éprouver un scorer	Écrire une assertion, la faire échouer sur un défaut contrôlé, puis la corriger ; calibrer un juge LLM par une lecture humaine
Expliquer une preuve de gouvernance	Montrer qu'une merge request est réellement bloquée par l'évaluation, puis débloquée par une correction, et dire où chaque contrôle s'applique

Modalités pédagogiques

Support de cours, guides pas à pas en ligne et dépôt de travaux pratiques fournis. Alternance d'apports théoriques et de travaux pratiques autour d'un produit fil rouge unique, construit puis évalué et gouverné sur les deux jours. Les 14 heures couvrent le parcours essentiel de chaque TP ; les approfondissements sont indiqués dans les guides. Programme sur-mesure, adaptable au contexte et à la stack du groupe.